

Word Frequency

Documentação técnica completa do projeto: da especificação inicial à entrega das seis fases de desenvolvimento, dos dois recursos bônus (gráfico e análise por URL) e da landing page explicativa adicionada posteriormente.

Tier 1 · Iniciante

HTML · CSS · JavaScript

Node.js + Express

+ Landing Page

Documento gerado em	Status
31 de agosto de 2026 (revisão 2)	Todas as fases concluídas + landing page

Visão Geral do Projeto

O que é o Word Frequency e qual problema ele resolve.

O Word Frequency é uma aplicação classificada como Tier 1 (Iniciante), cujo objetivo é calcular a frequência de palavras em um bloco de texto. O usuário informa um texto — digitado ou colado, com até 2048 caracteres — clica no botão Translate e recebe uma tabela com cada palavra única encontrada e o número de vezes que ela aparece, ordenada da mais para a menos frequente.

Esse tipo de cálculo é a base do modelo Bag of Words, técnica amplamente usada em busca, ordenação e análise semântica de texto — o projeto serve como uma introdução prática a esses conceitos.

Além do escopo obrigatório, o projeto foi estendido com dois recursos bônus previstos na especificação original: visualização gráfica dos dados e análise de texto extraído diretamente de uma URL.

Posteriormente, uma landing page foi criada para apresentar a funcionalidade do projeto antes de o visitante abrir a aplicação (ver seção 8).

Origem da especificação: o projeto partiu de um documento de especificação (App.md) contendo as histórias de usuário obrigatórias e os recursos bônus sugeridos. A partir dele foi criado um guia de desenvolvimento (CLAUDE.md) definindo stack, fases, estrutura de pastas, comandos e convenções de código — usado como referência durante toda a implementação.

Funcionalidades

Tudo o que foi especificado — e o que foi entregue.

Obrigatórias — 100% ENTREGUE

- ✓ Caixa de texto, botão "Translate" e tabela de frequência visíveis na tela
- ✓ Entrada de texto grande, com limite de 2048 caracteres
- ✓ Análise disparada ao clicar em "Translate"
- ✓ Mensagem de erro quando o campo está vazio
- ✓ Tabela populada com palavra + contagem de ocorrências
- ✓ Tabela ordenada em ordem decrescente de frequência

Bônus — 100% ENTREGUE

- ✓ Representação gráfica da frequência (column chart com Chart.js)
- ✓ Análise de conteúdo a partir de uma URL informada pelo usuário, via backend dedicado

Refinamentos de UX adicionados — durante a fase de polimento, foram incorporados tratamentos que não estavam explícitos nas histórias de usuário, mas evitam experiências confusas:

- Mensagem dedicada para quando o texto é válido mas não contém nenhuma palavra reconhecível (ex.: apenas pontuação)
- Aviso visual quando o texto se aproxima do limite de 2048 caracteres
- Feedback visual (animação) ao clicar em "Translate" e ao inserir novas linhas na tabela
- Suporte a tema claro/escuro automático, seguindo a preferência do sistema
- Atributos de acessibilidade (`role="alert"`, `role="status"`) para leitores de tela

Stack Tecnológica

Ferramentas escolhidas e o motivo de cada escolha.

CAMADA	TECNOLOGIA	MOTIVO
Front-end	HTML + CSS + JavaScript vanilla	Aplicação simples, sem estado complexo — dispensa framework e build step nas fases 1 a 4.
Gráfico	Chart.js (via CDN)	Biblioteca leve, cobre bar/column chart sem exigir bundler.
Backend	Node.js + Express	Necessário apenas para buscar conteúdo de URLs externas, evitando bloqueio de CORS no navegador.
Extração de HTML	cheerio	Parsing e remoção de tags irrelevantes (script/style) para extrair texto limpo da página.
Testes	node:test (nativo)	Runner de testes embutido no Node — zero dependências extras para validar a lógica pura.
Landing page	HTML + CSS (self-contained)	Página explicativa estática, sem dependências novas — reutiliza os tokens de cor do style.css do app.
Persistência	Nenhuma	Toda a contagem roda em memória, no cliente — não há necessidade de banco de dados.

Fases de Desenvolvimento

Do mais simples ao mais complexo — todas concluídas.

1 Estrutura estática

CONCLUÍDA

Layout inicial da aplicação, sem lógica de negócio.

- Textarea, botão "Translate" e tabela vazia
- Limite de 2048 caracteres via atributo maxlength + contador visual

2 Lógica de contagem (core)

CONCLUÍDA

O coração do projeto: transformar texto em contagem de palavras.

- Tokenização com normalização de caixa e remoção de pontuação
- Contagem de ocorrências em função pura, sem tocar no DOM
- Renderização inicial da tabela, ainda sem ordenação

3 Ordenação e validações

CONCLUÍDA

Fechamento de todas as histórias de usuário obrigatórias.

- Ordenação decrescente por frequência
- Mensagem de erro para campo vazio
- Tratamento de espaços múltiplos, quebras de linha e acentuação

4 Polimento de UX

CONCLUÍDA

Transformar a aplicação funcional em uma experiência agradável.

- Responsividade (mobile e tabelas com rolagem horizontal)
- Feedback visual em cliques e inserção de linhas
- Tratamento do caso de texto sem palavras reconhecíveis
- Acessibilidade com atributos ARIA

5 Bônus: gráfico

CONCLUÍDA

Representação visual da frequência de palavras.

- Column chart com Chart.js, limitado às 15 palavras mais frequentes para manter a leitura clara
- Cores do gráfico acompanham o tema claro/escuro da aplicação

6 Bônus: análise via URL

CONCLUÍDA

Permite analisar o conteúdo de uma página web em vez de texto colado manualmente.

- Alternância entre modo "Texto" e modo "URL" na interface
- Backend Express com endpoint /api/extract que busca e limpa o HTML da página
- Tratamento de erros: URL inválida, falha de rede, conteúdo não-HTML

Arquitetura e Estrutura

Como o código está organizado e por quê.

A separação de responsabilidades é o princípio central da arquitetura: a lógica de contagem de palavras é pura — recebe texto e devolve dados, sem depender do navegador — o que a torna simples de testar isoladamente. A manipulação da interface fica isolada em outro módulo, que apenas consome essas funções puras. A landing page (seção 8) é um documento estático independente, sem lógica própria além de links.

```
World_free/Primeiro Projeto/
├─ App.md                Especificação original (user stories)
├─ CLAUDE.md             Guia de stack, fases e convenções
├─ index.html            Entry point da aplicação
├─ landing.html          Landing page explicativa do projeto
├─ package.json          Configuração dos testes
├─ css/
│   └─ style.css         Estilos, responsividade e temas
├─ js/
│   ├── main.js          Eventos, DOM e orquestração
│   ├── wordFrequency.js  Lógica pura: tokenizar, contar, ordenar
│   └─ chart.js          Renderização do gráfico (Chart.js)
├─ tests/
│   └─ wordFrequency.test.js Testes automatizados da lógica pura
└─ server/               Backend do bônus "análise via URL"
    ├── index.js          Endpoint /api/extract
    └─ package.json
```

Fluxo de dados: `main.js` lê o texto do `textarea` (preenchido manualmente ou via URL) → chama `countWordFrequency()` e `sortByFrequency()` de `wordFrequency.js` → renderiza a tabela e repassa o mesmo resultado para `renderChart()`, que desenha o gráfico. Nenhuma dessas funções de cálculo conhece a existência do DOM. `landing.html` não participa desse fluxo — é o ponto de entrada explicativo que leva o visitante até `index.html`.

Testes Automatizados

Cobertura da lógica de negócio, sem dependências externas.

A lógica de tokenização, contagem e ordenação é validada por 8 testes automatizados, executados com o runner nativo do Node.js (`node:test`) — sem necessidade de instalar frameworks de teste. Todos passam na versão atual do código.

FUNÇÃO TESTADA	CENÁRIOS COBERTOS
<code>tokenize()</code>	Divisão e normalização de palavras; remoção de pontuação; texto vazio; múltiplos espaços e quebras de linha
<code>countWordFrequency()</code>	Contagem de palavras repetidas; mapa vazio para texto vazio
<code>sortByFrequency()</code>	Ordenação decrescente com empates; lista vazia para mapa vazio

Resultado da última execução: 8 de 8 testes aprovados (`npm test`). A landing page não introduziu lógica testável — é HTML/CSS estático, sem funções puras novas para cobrir.

Como Executar

Passo a passo para rodar o projeto localmente.

Front-end (fases 1 a 5) e landing page

```
npx serve .  
# ou  
python3 -m http.server 8000
```

Com o servidor no ar, abra `http://localhost:8000/landing.html` para a página explicativa ou `http://localhost:8000/index.html` para a aplicação em si.

Rodar os testes

```
npm test
```

Backend do bônus "análise via URL" (fase 6)

```
cd server  
npm install  
npm start
```

Com o backend em execução em `http://localhost:3000`, a opção "URL" na interface passa a buscar e analisar o conteúdo de qualquer página informada.

Landing Page

Página explicativa criada para apresentar o projeto antes da aplicação.

Foi adicionado um novo arquivo, `landing.html`, na raiz do projeto: uma página de apresentação que explica a funcionalidade do Word Frequency para quem ainda não abriu a aplicação — hero com chamada para ação, passo a passo de uso, grade de funcionalidades, stack técnica e um CTA final, todos levando para `index.html`.

Estrutura da página

- **Hero** — título, resumo do que o app faz, botões "Experimentar agora" e "Ver especificação", e um preview estático da tabela de frequência
- **Como funciona** — 3 passos (colar texto/URL → clicar em Translate → ver tabela e gráfico)
- **Funcionalidades** — grade com as 6 principais capacidades do app, incluindo os dois recursos bônus
- **Stack técnica** — chips com as tecnologias usadas no projeto
- **CTA final** — segundo convite para abrir a aplicação

Design guiado pela skill ui-ux-pro-max — o visual foi revisado com a skill de design ui-ux-pro-max (base local de padrões de UI/UX), consultada nos domínios `--design-system`, `landing`, `icons` e `ux`. Decisões aplicadas:

Padrão de layout	Feature-Rich Showcase (hero → grade de funcionalidades → CTA), identificado como o melhor encaixe entre os padrões de landing da base de dados.
Tipografia	Par sugerido para ferramentas de desenvolvedor: JetBrains Mono (títulos e tags) + IBM Plex Sans (corpo), reforçando o tom técnico do produto.
Ícones	Emojis substituídos por SVGs outline (estilo Phosphor), marcados <code>aria-hidden</code> por serem decorativos ao lado de texto visível — anti-padrão evitado.
Acessibilidade & interação	Estados de foco visíveis em todos os links/botões, alvos de toque com no mínimo 44px, e suporte a <code>prefers-reduced-motion</code> .

A paleta de acento azul do `style.css` existente foi mantida (light/dark aware) em vez da paleta genérica sugerida pela busca — decisão de consistência de marca com o `index.html` real, não uma recomendação direta da skill.

Considerações Finais

Status atual e possíveis evoluções futuras.

O projeto Word Frequency está com todas as histórias de usuário obrigatórias e os dois recursos bônus da especificação implementados e validados. A aplicação roda inteiramente no navegador para o fluxo principal, com um backend leve e opcional apenas para o recurso de análise por URL. A landing page complementa o projeto como ponto de entrada explicativo, sem alterar a lógica ou os testes existentes.

Possíveis evoluções (fora do escopo original)

- Deploy do front-end — landing page + app (ex.: GitHub Pages) — e do backend (ex.: Render, Railway)
- Lista de stop words configurável, para ignorar palavras muito comuns
- Exportação da tabela de frequência em CSV
- Histórico de análises anteriores (exigiria persistência)